

Lab 1: Architectural Design and Requirement Analysis

18-449/649 (Due September 17th, 2026 at 11:59pm)

Introduction

The automotive industry has seen a steady increase in complexity and capability of system requirements over time. Cars have shifted from almost exclusively mechanical systems to ones that are highly computerized with mechanical components being replaced with digital interconnects. Drive-by-wire is a great example where now the steering, throttle, and brake controls are completely digitized with sensors sending signals through computers that drive actuators without any mechanical backups. This allows for increased functionality, but also poses new challenges in the embedded domain where safety critical operation is essential and resources are relatively scarce. In response, the automotive industry has gone through a transformation from signal oriented system design to domain-specific and now “zone” architectures. This represents a significant shift on how we design, test, and even reason about distributed embedded system design.

The goal of this project is to design the electronics and embedded software for a drive-by-wire system that is able to take inputs from an automotive cockpit and transmit them to table-top steering and drivetrain assembly. As shown in Figure 1, the linkage from the cockpit to the vehicle body needs to be bi-directional such that: (1) when the steering wheel moves, the wheels on the vehicle correspondingly move, and (2) if force is detected on the vehicle wheels then that force is applied as haptic feedback on the steering wheel. Throttle and brake are unidirectional in that the pedals simply control the drive motors. The throttle should set a velocity proportional to pedal position that is maintained using encoder feedback.

System Overview

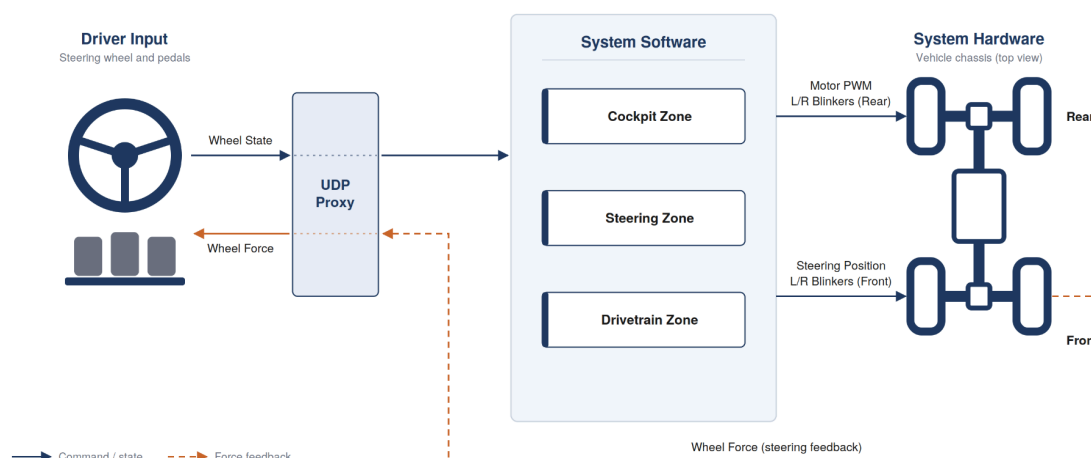


Figure 1: System Overview Diagram

In the next sections, we will discuss three main required subsystems (zones) that your teams will be required to implement for the project. Teams are free to add additional zones as needed, but at least three zones must interact over a Controller Area Network (CAN) bus. CAN is the required main communication bus for the system, and it must be implemented as a dual-redundant bus: two independent CAN buses (CAN A and CAN B) connect every zone, and all inter-zone traffic (control commands, force feedback, heartbeats, and error notifications) must be carried on both buses so that the system continues to operate normally if either bus, or any single node's connection to either bus, fails. See requirement R4 for the detailed bus and redundancy requirements. Note that we require a test input on each zone such that a single press simulates a zone failure, and a double press (two presses within 500ms) restores the zone. A test button failure must be local to the particular zone (the failed zone stops transmitting on the CAN bus and drives its outputs to their safe state), but the system response must be global: the remaining zones must detect the failure (e.g. through a missed CAN heartbeat) and enter the system-wide error state defined in R3.

Each group will be provided with an RPI, car chassis, servo, motor with encoder. Your team will be required to build a PCB controller board that connects to the RPI. The RPI can act as your cockpit zone, but the controller board needs to have two STM32 MCUs that act as the other two zones (steering and drivetrain). Each of the three zones must have two independent CAN interfaces (its own CAN controller and CAN transceiver for CAN A, and a second controller and transceiver for CAN B), and all three zones must be connected to both properly terminated buses. Each node must also provide fault-injection jumpers that allow that node to be individually disconnected from CAN A, from CAN B, or from both (see R4.8). Note that neither the RPI nor the STM32F401RE includes an on-chip CAN controller, so two CAN controllers per zone (e.g. SPI-attached MCP2515/MCP2518FD devices, or a dual-channel CAN HAT for the RPI) or a CAN-capable MCU variant approved by course staff is required.

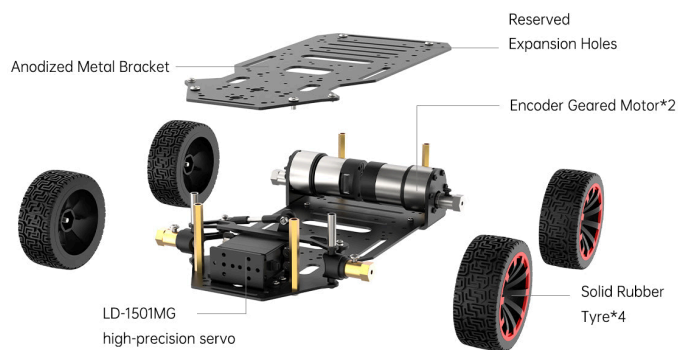
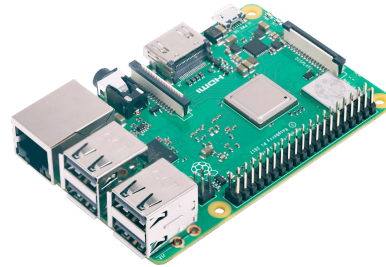
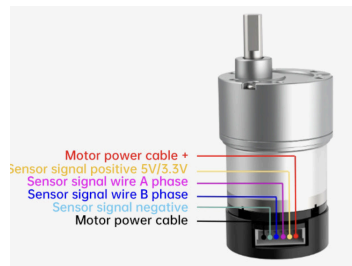


Figure 2: Provided Components

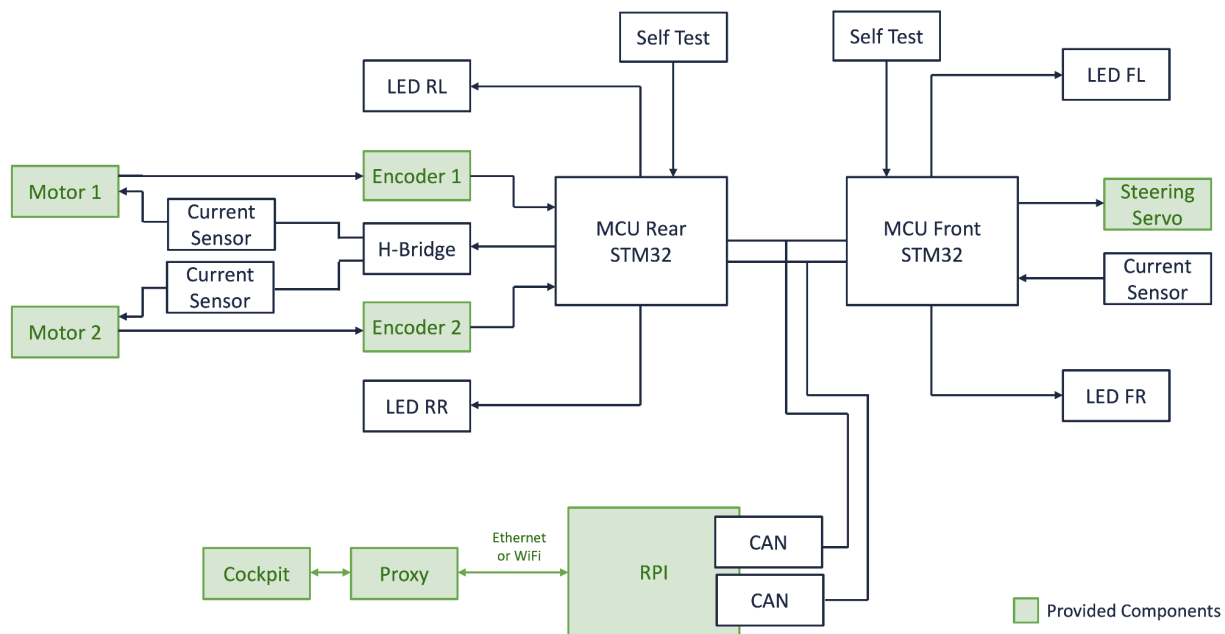


Figure 3: Hardware Block Diagram

Cockpit Zone

The cockpit zone provides the interface between the steering wheel and the rest of the automotive system. The majority of inputs to this zone come from a commercial (game) controller, including the steering, throttle, brake, left turn, right turn, and testing. The only software input is obtained through a UDP proxy stream, which specifies how much force feedback should be exerted on the steering wheel. The cockpit zone acts as the gateway between the UDP proxy and the CAN bus: it must translate each received UDP state update into CAN control messages for the steering and drivetrain zones (transmitted on both CAN A and CAN B), and translate force-feedback CAN messages received from the steering zone on either bus into the UDP force message sent back to the proxy.

In addition to the force feedback sent to the steering wheel, the output should include a set of test points hooked up to the UDP input and output stream. These test points will be used by course staff to evaluate your zone functionality. The only custom hardware required for this zone is the dual CAN interface between the RPI and your main controller board that contains the other two zones (two CAN controllers plus transceivers attached to the RPI, or an equivalent dual-channel CAN HAT), including the per-bus isolation jumpers required by R4.8. The cockpit zone does not need a physical self-test button; the "testing" button on the game controller serves as its test input and must follow the same single-press fail / double-press restore behavior as the other zones. At least two RPI GPIO pins must be exposed as test points and toggled by software: one on reception of each UDP state update packet, and one on transmission of the corresponding CAN control message. These edges define the start of the

response-time measurements in R2. If no UDP state update is received for 100ms, the cockpit zone must treat itself as failed (stop transmitting its heartbeat) until the stream resumes.

For UDP protocol specification, see appendix B.

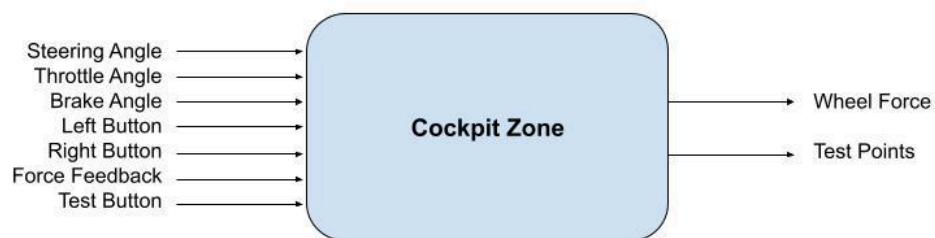


Figure 4: Cockpit Zone Input/Output on RPI

Front (steering) Zone

The steering zone provides an interface between the steering wheel and the front of the automotive system, including an actuator (e.g. servos, stepper motors) for the steering position and two blinkers for left and right turn signals. Upon receiving a CAN control message from the cockpit zone, the steering zone needs to extract the commanded position (angle) of the actuator and the left/right blinker on/off signals, then drive the actuator or blinkers correspondingly. The mapping from steering-wheel input value to actuator angle must be monotonic, cover the full mechanical range of the chassis steering, and be documented in your requirements document.

In addition, the steering zone needs to measure the load on the steering actuator (e.g. servo motor current through a sense resistor, or commanded-versus-measured position error) and transmit it periodically on the CAN bus as a signed force value in the range -100 to 100 (matching the UDP encoding in Appendix B). The cockpit zone forwards this value to the proxy to set force feedback on the steering wheel. The measurement method and the mapping from measured load to force value must be documented.

The steering zone must provide a set of test points that allow for timing verification of each subsystem with an oscilloscope. For the steering zone, the test points must cover the front left and right blinker outputs, the servo control signal, a GPIO toggled on CAN control message reception, and a GPIO toggled on CAN force message transmission.

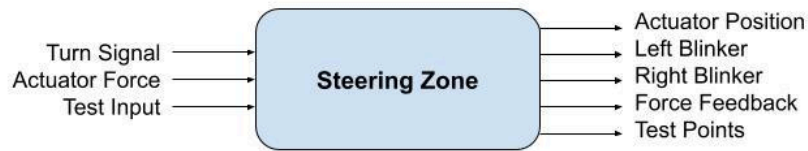


Figure 5: Steering Zone Input/Output

Rear (drivetrain) Zone

The rear drivetrain zone provides an interface between the steering wheel and the back of the automotive system, including PWM motors and rear blinkers. Upon receiving a CAN control message from the cockpit zone, the drivetrain zone needs to extract the throttle, brake, and turn signals, then drive the PWM motor or blinkers correspondingly. The throttle position must map to a target wheel velocity through a documented, monotonic mapping (with full throttle corresponding to a documented maximum velocity), and the target velocity must be maintained by closed-loop control using encoder feedback. Wheel velocity for control purposes is the average of the two encoder velocities. The motor current sensors shown in Figure 3 are used to check for disconnected (open) or overloaded (high-current) motor failures. A "stuck wheel" is defined as a non-zero commanded velocity with no encoder transitions for at least 200ms; in this condition the controller must increase the PWM duty-cycle up to a documented maximum until the wheel moves.

The drivetrain zone does not need to send any data back to the UDP proxy, but like every zone it must transmit a periodic heartbeat message on the CAN bus (see R4).

The test points must be hooked up to all output signals shown in the figure below (motor PWM, H-Bridge direction inputs, and rear left and right blinkers), plus a GPIO toggled on CAN control message reception.

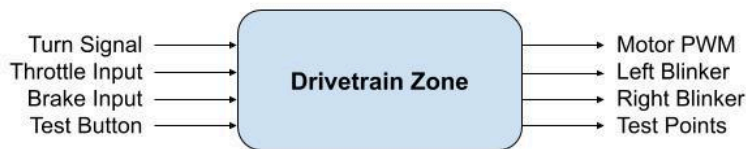


Figure 6: Drivetrain Zone Input/Output

Use Case Scenarios

S1: Throttle Control

- Motor Velocity is set proportional to the Throttle Pedal angle when brake not pressed

S2: Brake Control

- Motor PWM halted and H-Bridge placed in dynamic braking mode when Brake depressed
- Brake takes priority over Throttle under all circumstances

S3: Steering Wheel

- The steering actuator needs to reflect the angle of the cockpit steering wheel input
- The force on the steering actuator should impart a proportional force to the steering wheel for user feedback

S4: Turn Signal and Hazard Blinkers (basic)

- When the right blinker button is pressed, the front and rear right blinkers should blink and the left blinker should stop
- When the left blinker button is pressed, the front and rear left blinkers should blink and the right blinker should stop
- When the right blinkers are blinking and the steering wheel passes a right turn threshold and then the wheel returns past the right turn threshold, the blinkers stop
- When the left blinkers are blinking and the steering wheel passes a left turn threshold and then the wheel returns past the left turn threshold, the blinkers stop
- In the system error state, all four blinkers flash in unison at 2Hz (+/- 10%), 50% duty-cycle, indicating a hazard

S5: Self Test

- Each zone will have a self-test button that disables that zone. A single press causes the zone to fail (it stops transmitting heartbeats on both buses), while a double press restarts the zone
- The fail safe state of the system will be to disable the Motor PWM output, engage the H-Bridge brake, and triggers a system-wide error that enables the hazard signal

S6: CAN Bus Fault Tolerance

- When any single node is disconnected from CAN A (or from CAN B) using its isolation jumper, the system continues to operate with no loss of function and all R2 response times still met; the affected node and bus are reported as degraded
- When an entire bus (CAN A or CAN B) is lost, e.g. by disconnecting every node from it, the system continues to operate normally on the remaining bus
- When a node is disconnected from both CAN A and CAN B, the remaining zones treat it as a failed zone and enter the system error state (S5); the isolated node itself enters its fail-safe state

- When the jumper(s) are restored, the degraded or failed condition clears and the system returns to normal operation without a power cycle

Requirements and Constraints

The project is governed by a number of requirements and constraints that define viable design options. Each requirement below must be demonstrated through the entire design process with unit testing, validation and traceability to your code. Constraints consist of a mixture of functional and non-functional attributes that need to be met to receive full credit on the lab. In this document "must" denotes a mandatory requirement and "should" denotes a recommendation. Where a value is left to the team (thresholds, mappings, message IDs), it must be recorded in the requirements document and used consistently in the design, tests and demo.

R1: Zone Architecture

The system must have at least three zones (cockpit, steering, drivetrain) connected by the dual-redundant CAN buses defined in R4. The two STM32 zones (steering and drivetrain) must run a priority-based preemptive real-time operating system (**Zephyr**, Mynewt, FreeRTOS, etc). Each zone is defined by a unique microcontroller with its own independent clock source (its own crystal or oscillator; no clock signal may be shared between zones).

R2: Response Times

All response times in R2 are measured end-to-end: from the cockpit zone GPIO edge that marks reception of the UDP state update packet from the proxy, to the corresponding output change on the STM32 zone, and therefore include CAN transmission latency.

2.1 Throttle: Upon reception of a periodic UDP state update (including throttle) message from the cockpit proxy, the motor PWM duty-cycle must change to the new value corresponding to the throttle position within **2ms**.

2.2 Brake: Upon reception of a periodic UDP state update (including brake) message from the cockpit proxy, the motor must enter brake mode (H-Bridge in dynamic braking, PWM disabled) within **2ms**.

2.3 Steering: Upon reception of a periodic UDP state update (including steering wheel) message from the cockpit proxy, the servo control signal must update to the new commanded angle within **50ms**. The steering zone must transmit a force feedback CAN message at least every 50ms, and the cockpit zone must send a UDP force message to the proxy based on the most recent force value every **50ms**.

2.4 Turn Signal: Upon reception of a periodic UDP state update (with left or right blinker button) message from the cockpit proxy, the left (or right) blinker must start blinking within **100ms**. State changes (on-to-off and off-to-on edges) of the front and rear blinkers on the same side must be synchronized to within **1ms**. (note: two free-running 1Hz timers on independent clocks may not

meet this; a synchronization strategy over CAN is required). When active, the blinkers must blink at a 50% duty-cycle at a rate between 0.9Hz and 1.1Hz (nominal 1Hz).

2.5 Test Button and Error Propagation: All response times must be measurable externally (i.e. with an o-scope) using the test points defined for each zone. Upon a single press of a zone test button, that zone must enter its local fail state within **10ms**. All other zones must detect the failure and enter the system-wide error state (hazard flashing, motor braked) within **100ms** of the button press. Test button inputs must be debounced; presses separated by less than 50ms are treated as a single press.

R3: Error State

The system enters the error state when any zone fails its self-test, when any zone misses three consecutive heartbeat messages from another zone, or when a zone receives an out-of-range or malformed control value. Upon entering the system error state, the motor PWM must be disabled, the H-Bridge must apply dynamic braking, all four blinkers must flash the hazard pattern (S4), and the steering zone must continue to track the steering wheel unless the steering zone itself is the failed zone. Each STM32 zone must have a hardware test push button, and the cockpit zone must use the game controller test button, such that a single press fails that zone and thereby triggers the system-wide error state. A double press of the same test input (two presses within 500ms) must restore the failed zone; the system must return to normal operation within 500ms of the restore, once heartbeats from all zones are again being received. On power-up every zone must start in the error state and may transition to normal operation only after it has received heartbeats from all other zones on at least one bus and, for the STM32 zones, a valid control message. The error state must be entered and left deterministically and must be documented in the system state chart.

R4: Communication Bus (CAN)

4.1 Bus: The main (and only) communication path between zones must be a dual-redundant Controller Area Network (CAN) consisting of two physically independent buses, CAN A and CAN B, each compliant with ISO 11898-2 (high-speed CAN, classic CAN 2.0A or 2.0B frames). Each zone must be attached to each bus through its own dedicated CAN controller and CAN transceiver (two controllers and two transceivers per zone); the two interfaces of a zone must not share a transceiver, a controller, or any bus wiring, so that a fault on one bus cannot affect the other. Point-to-point links (UART, SPI, I2C, GPIO handshakes) between zones are not permitted; SPI (or similar) between a microcontroller and its own CAN controller is internal to that zone and is allowed.

4.2 Bit rate: Both buses must operate at a nominal bit rate of 500 kbit/s. All nodes on both buses must use the same bit timing; the timing must be documented.

4.3 Physical layer: Each bus must be a single linear bus with a 120 Ohm termination resistor at each of its two ends (and only at the ends). Termination must be selectable (jumper or footprint) on the PCB and must be connected on the bus side of the node isolation jumpers (R4.8), so that isolating a node never removes a bus termination. All nodes must share a common ground reference, and each bus's CAN_H / CAN_L must be routed as a differential pair, physically separated from the other bus. The RPI must be connected to both CAN A and CAN B; it may not be attached to only one bus or to a bus of its own. Bus A and bus B must use distinct, clearly labeled connectors and wire colors.

4.4 Message catalog: Every CAN message used by the system must be listed in a message catalog in the requirements document with its identifier, transmitting zone, receiving zone(s), payload layout (byte/bit, units, range, scaling), transmission period or trigger, and priority rationale. Safety-relevant messages (brake, error) must have higher priority (lower identifier) than convenience messages (blinkers, force feedback). Control messages from the cockpit zone must be transmitted on every UDP state update received from the proxy and no less than every 50ms. Every message in the catalog must carry a sequence counter (at least 4 bits) so that duplicate copies received on the two buses can be identified (R4.7).

4.5 Heartbeat and fault detection: Each zone must transmit a heartbeat message every 20ms (+/- 10%) on both buses while operating normally. A zone that fails (test button, watchdog reset, or internal fault) must stop transmitting its heartbeat on both buses; each STM32 zone must enable its independent watchdog. Every zone must monitor the heartbeats of the other zones on each bus independently. A zone is considered failed, and the system-wide error state (R3) must be entered, only when three consecutive heartbeats from that zone are missed on both buses. Missing heartbeats on only one bus indicates a bus or connection fault and is handled by R4.7, not as a zone failure. Zones must also treat a received control value outside the documented valid range as an error.

4.6 Bus utilization and observability: Worst-case utilization of each bus at the required message rates must be calculated in the timing analysis and must not exceed 50% on either bus. Each zone must expose CAN_H and CAN_L of both buses on a test header so either bus can be observed with an oscilloscope or logic analyzer, and each zone must toggle a GPIO test point on CAN message transmission and reception so bus latency can be measured.

4.7 Redundancy management: The redundancy scheme is hot/parallel: every message must be transmitted on both CAN A and CAN B at the same time, and a receiver must accept the first valid copy that arrives on either bus and discard the later duplicate using the message sequence counter (R4.4). The system must meet every response time in R2 with either bus, or any single node's connection to either bus, absent; no failover or switchover delay is permitted, since both buses are always active. Each zone must track the health of each of its bus connections: a bus connection is declared degraded when no traffic from any other node has been received on that bus for three heartbeat periods (60ms) while traffic is still being received on the other bus. A degraded bus must be indicated on the affected node by a per-bus status LED and reported in that node's heartbeat payload so the cockpit zone can log it; the system

must remain fully operational (no hazard, no braking) while degraded. When traffic on the degraded bus resumes, the degraded indication must clear within 100ms. A zone that receives no heartbeats from any other zone on either bus for three heartbeat periods must conclude that it has been isolated from the network and must enter its own fail-safe state (motor braked / steering held, hazards on) within 100ms.

4.8 Fault-injection jumpers: Every node (including the RPI cockpit zone) must provide isolation jumpers that allow course staff to disconnect that node individually from CAN A, from CAN B, or from both buses while the system is powered and running. Each isolation jumper must disconnect both CAN_H and CAN_L of that node from the bus (e.g. a 2x2 header with two shunts per bus, or a 2-position DIP switch per bus), must be located between the node's transceiver and the bus connector, and must not affect bus termination (R4.3) or any other node. Jumpers must be clearly labeled (e.g. "CAN A" / "CAN B", node name) and reachable without disassembling the vehicle. The transceivers must tolerate being connected and disconnected from a live bus, and a CAN controller that enters bus-off must recover automatically within 100ms of the bus being restored. The following fault scenarios must be supported and demonstrated: (a) any single node isolated from CAN A only; (b) any single node isolated from CAN B only; (c) two different nodes isolated from different buses simultaneously (e.g. steering off CAN A and drivetrain off CAN B), during which all three zones must remain fully operational; (d) any single node isolated from both buses, which must produce the system error state within 100ms per R2.5; (e) all nodes isolated from the same bus (complete bus loss); and (f) restoration of any jumper, after which normal operation must resume within 500ms without a reset or power cycle. The test document must include a fault-injection matrix listing each scenario, the expected state of every zone and every status LED, and the measured recovery time.

Lab 1 Deliverables

Design Documents (30 points)

- Requirement Document
 - Introduction with system objective
 - System state chart
 - Sequence diagrams for each use-case (S1-S6)
 - Traceability for each use-case (S1-S6)
 - Proposed timing analysis
 - CAN message catalog (R4.4) and bus utilization calculation (R4.6)
 - Table of all team-defined values (turn thresholds, throttle-to-velocity mapping, force mapping, valid ranges)
 - Dual-CAN redundancy design: bus topology showing both buses, termination, isolation jumpers and per-bus status LEDs on every node (R4.1, R4.3, R4.8)
 - Fault-injection test matrix covering scenarios (a)-(f) of R4.8

Unit Test and Traceability (5 points)

- Testing document

Final Demonstration and Report (just for reference, not due as part of Lab 1)

System Functionality (50 points)

- Demonstrate the system to staff

Time Analysis and Test (5 points)

- Demo with o-scope
- State transition diagram with response times

Final Report (10 points)

- Requirement Design Document (updated as needed)
- Implementation details (HW / SW)
- Annotated photos of final platform

Appendix A: Unit Testing

Your testing document and plan should include a list of tests for the blinker subsystem and then a system of your own choice. In the document, make sure to describe how you are testing each component and the expected results.

Requirements:

Each team will have to demonstrate comprehensive testing of at least the following subsystems:

1. Blinker subsystem
2. Your choice of any other interesting subsystem (motor control, steering+haptic feedback, custom subsystem etc.)
3. CAN bus redundancy, using the isolation jumpers to execute every fault-injection scenario in R4.8

Define a set of scenarios with several test cases.

Demonstrate the functional correctness of system-under-test for the three subsystems.

Example Scenarios for Blinker Subsystem (generate your own table for your custom subsystem):

Precondition	Input	Postcondition
Left blinker is on. Wheel is NOT past left threshold	Wheel turns left past threshold	Left blinker is still on.
Left blinker is on. Wheel is past left threshold	Wheel returns from threshold.	Left blinker turns off.
Right blinker is on Wheel is in NOT past right threshold	Wheel turns right past threshold	Right blinker is still on.
Right blinker is on Wheel is past left threshold	Wheel returns from threshold	Right blinker turns off.
Left blinker is off	Press left turn button	Left blinker is on.
Right blinker is off	Press right turn button	Right blinker is on.
-	Wheel values out of range	Error state (Self defined)
-	Click failure button	Hazard lights are on.
Any zone is in the fail state Hazard lights are on	Double click failure button	Hazard lights are off.
Right blinker is on Wheel is past right threshold	1.Press right turn button 2.Wheel returns from right threshold to left threshold	Self-defined deterministic state

Deliverable:

- Demo the unit tests to TAs
 - Include custom test cases for your subsystem of choice
 - Demonstrate several test cases for each scenario showing reasonable coverage
- Submit a document including the test cases/scenarios for each unit test case.

Testing Ideas:

There is no one-size fits all testing method that will work for every possible project. You will need to work within the scope of your system to define an appropriate set of tests. Some general hints include:

- Make it easy to enable/disable unit testing mode at the software level. In testing mode, introduce a way to generate precondition states and inputs such as:
 - Simulating UDP packets from the proxy and injecting or logging CAN frames on either bus (e.g. with a USB-CAN adapter)
 - Using the R4.8 isolation jumpers to remove nodes from CAN A, CAN B, or both, and verifying the degraded indication, duplicate suppression, and recovery timing with an oscilloscope on the status LEDs and CAN test headers
 - Include tests in the source code
 - Loading and running tests from an SD card/storage
 - Use tools such G-Test for systematic testing of software components
 - Consider using automatic test pattern generation strategies
- Briefly discuss coverage tests during weekly meetings prior to the demo with your TA to identify reasonable coverage for each scenario

Appendix B: Logitech Steering Wheel Module (Cockpit Control)

Github page: <https://github.com/arjunr2/logitech-wheel-dev>

This module allows access from Python to the steering wheel SDK provided by Logitech for their gaming steering wheels. The API of this module is the same as the "Logitech Gaming Steering Wheel SDK" version 8.75.30. This SDK and it's documentation can be downloaded from the [Logitech G website](#). The module has been tested on Python 3.8 and only works on Windows.

This module includes part of the before mentioned steering wheel SDK (the `LogitechSteeringwheel.dll` and `LogitechSteeringWheelEnginesWrapper.dll` files). These files do not fall under the open source license with the rest of the module. They are re-distributed with permission from Logitech (also see the README file in the SDK). Therefore, there is no need to obtain the SDK. The only dependency of this module is the Logitech Gaming Software.

Installation instructions

The `logitech-steering-wheel` module is available from PyPi and can be installed using pip (`pip install logitech-steering-wheel`). The only dependency is the Logitech Gaming Software that can be downloaded for [Logitech's website](#). The module has been tested with version 5.10. This version works with older steering wheels. According to Logitech, the SDK is also compatible with newer versions of the gaming software.

In some cases, Windows has been known to install other drivers when a steering wheel is plugged in. These default drivers installed by Windows do not work with the SDK. The solution to this problem is to:

1. plug in the steering wheel,
2. uninstall the driver Windows automatically installed, and leave steering wheel plugged-in and turned on, and
3. Install the Logitech Gaming Software (this should automatically install the correct drivers)

Using the SDK

For quick access, the SDK documentation is included in the repo. The respective Python functions to access DLL functionality can be found [here](#). Refer to the script [proxy_gui.py](#) for details on interfacing with the wheel.

Proxy Application Test

You will require 2 machines (call them **W** and **R**), with at least 1 running Windows (**W**) to interface with the logitech wheel. Make sure **W** and **R** can communicate over either wireless or Ethernet.

Setup

1. Plug in the steering wheel to **W** after performing the installation instructions above successfully. Make sure the wheel powers on.
2. Configure the IP Addresses and Ports accordingly in applications `proxy_gui.py` and `proxy_receiver/receiver.c`.
3. Compile C receiver on **R**: `cd proxy_receiver; gcc -pthread receiver.c -o proxy_receiver`

Running the applications

1. Run `python3 proxy_gui.py` script on **W**
2. Run `./proxy_receiver` on **R**

You should see wheel state reception on **R** and force feedback data on **W**. The data rates on either side can be set by changing the `TX_INTERVAL_MS` field

UDP Data Encoding

The following section highlights the data encoding for the current communication protocol in the test application. This was designed to be simple enough to enable meeting the basic requirements. You may add more functionality if you'd like

Wheel ---> System

The transmission from `proxy_gui.py` send the following:

1. 32-bit packet counter
2. [DIJOYSTATE2](#) ctype structure

These are both packed in order into a bytearray. Note, this enables convenient unpacking of wheel state into an object in C, as demonstrated in the test [receiver](#)

System ---> Wheel

The `proxy_gui.py` expect to receive:

1. A single byte representing the signed direction of force (valid values include -100 to 100 as highlighted in the SDK)